

Week 1: Programming Paradigm Analysis

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

July 11th, 2026

Part One: Code Analysis

All three programs perform the same task: they compute five statistics (student count, average, highest grade, lowest grade, and pass rate) from the same hardcoded list of ten grades. Because the task is identical, the structural and stylistic differences between the samples come entirely from the paradigm each one follows.

Imperative Version ([imperative.py](#))

Control flow. Everything happens as an explicit, ordered sequence of statements inside a single function. One for loop walks the grade list, and if statements inside it handle the highest, lowest, and passing checks. The program reads chronologically from top to bottom, and notably it computes all five statistics in a single pass over the data.

Data handling. The version is built on mutable state. Five accumulator variables (total, count, highest, lowest, passing_count) are initialized before the loop and updated in place on every iteration. The answer does not exist until the loop finishes, and correctness depends on the order in which state is mutated.

Abstraction. There is essentially none beyond the single function. Computation and printing are fused into one block, nothing is reusable or independently testable, and the passing threshold 70 appears as an unnamed magic number inside the loop. Stylistically this is the recipe style: transparent to read once, but with no nameable parts.

Functional Version ([functional.py](#))

Control flow. There are no explicit loops or conditionals at all. Iteration is implicit inside higher-order functions (filter, reduce, and map), and control flow is really expression evaluation and function composition: `generate_report` maps a list of functions over the data rather than executing steps in sequence.

Data handling. This is the opposite of the imperative version: there is no mutable state anywhere. The grades list is never modified, and every function is pure, taking data in and returning a new value out. Data flows through a pipeline of transformations instead of accumulating in variables.

Abstraction. Functions are the unit of abstraction and are treated as values: `calculate_count` aliases the built-in `len`, each statistic is a named lambda, and `create_analysis_functions` returns a list of functions that is mapped over the data. Each metric is a small, independently testable unit, and pure computation is cleanly separated from output (`print_report` is the only function with side effects). Stylistic quirks include `reduce(max, grades)` where plain `max(grades)` would do, `list(map(print, ...))` used purely for side effects, and an unused import of `partial`.

Object-Oriented Version ([objectoriented.py](#))

Control flow. Flow is organized as method calls on an object: the driver constructs a `GradeAnalyzer` and calls `print_results`, which orchestrates calls to the five getter methods.

Iteration still exists but is hidden inside built-ins within each method (sum, max, min, and a generator expression), so the list is traversed once per statistic.

Data handling. State is encapsulated rather than mutated or eliminated. The grade list is stored once in `self.grades` at construction, and every method derives its result on demand from that shared instance state. Data and the behavior that operates on it are bundled into one object.

Abstraction. The class is the abstraction boundary. Its public methods form an interface, so callers never see how a statistic is computed and the internals could be swapped without touching the driver code. The class is also the natural extension point for new statistics. Stylistically it uses Java-style `get_` prefixes rather than Pythonic properties, and `print_results` mixes presentation into an otherwise computational class.

Key Differences at a Glance

Dimension	Imperative	Functional	Object-Oriented
Where iteration lives	One explicit for loop	Hidden inside map, filter, and reduce	Hidden inside built-ins, once per method
State	Mutable accumulator variables	None; all functions are pure	Encapsulated in <code>self.grades</code>
Unit of abstraction	The statement; one large function	Small first-class functions	A class with a method interface
Passes over the data	One	One per statistic	One per statistic

Computation vs. output	Fused in one function	Fully separated (print_report is the only I/O)	Partially separated (print_results lives on the class)
------------------------	-----------------------	---	--

Two cross-cutting observations: first, all three samples hardcode the passing threshold of 70, but in three different places (an inline comparison, a named `is_passing` predicate, and inside a method body), which previews how differently each absorbs a requirements change. Second, the imperative version is the only single-pass implementation, which becomes relevant in the performance comparison in Part Four.

Part Two: Readability Assessment

The complete annotations appear as in-line comments in the three submitted files `imperative_modified.py`, `functional_modified.py`, and `objectoriented_modified.py`, where every strength is tagged [STRENGTH] and every weakness is tagged [WEAKNESS] directly beside the line it describes. The highlights are summarized below.

Imperative: Strengths

- Descriptive accumulator names (`total`, `count`, `highest`, `lowest`, `passing_count`) make each variable's purpose obvious.
- A single explicit loop means execution can be traced line by line with no hidden control flow.
- Derived values are computed immediately after the loop, close to where they are printed, so the reader never jumps around the file.

- f-strings with `:.1f` formatting keep the output code compact and readable.

Imperative: Weakness

- The passing threshold 70 is an unnamed magic number; a reader cannot tell whether it is policy or coincidence.
- Manual counting (`count += 1`) duplicates what `len(grades)` already provides, adding one more piece of mutable state to track.
- Computation and presentation are fused, so the statistics cannot be read, reused, or tested separately from the printing.
- Seeding highest/lowest from `grades[0]` silently assumes a non-empty list and would raise `IndexError` otherwise.

Functional: Strengths

- Every statistic is a small, named, single-purpose function, so the name itself documents the intent.
- All computation functions are pure, with no hidden state; each can be read and tested in isolation.
- The `is_passing` predicate gives the 70 threshold a name, unlike the other two versions.
- Pure computation is cleanly separated from I/O; `print_report` is the only side-effecting function.

Functional: Weaknesses

- Lambdas assigned to names (discouraged by PEP 8) lose docstrings and produce unhelpful tracebacks compared to `def`.
- Aliasing built-ins (`calculate_count = len`) forces the reader to chase a definition that turns out to be trivial.
- `reduce(max, grades)` obscures the simpler and universally recognized `max(grades)`.
- `count_passing` chains `filter` into `list` into `len`; a generator expression with `sum()` would state the intent directly.
- Returning a list of lambdas from `create_analysis_functions` is heavy indirection: the reader must mentally execute `map()` over functions just to know what the report says.
- `list(map(print, ...))` abuses `map` for side effects and builds a throwaway list of `None` values.
- `partial` is imported but never used, sending the reader hunting for a usage that does not exist.

Object-Oriented: Strengths

- The class name and method names together read like a table of contents for the file; the interface is self-documenting.
- Each statistic lives in its own short method built on a familiar built-in; every method body is one obvious line.
- The grade list is stored once at construction, giving every method a single source of truth.

- The generator expression in `get_pass_rate` counts matches idiomatically without an intermediate list.

Object-Oriented: Weaknesses

- The passing threshold 70 is again a magic number, buried inside a method body.
- Java-style `get_` prefixes are unidiomatic Python; properties would read more naturally.
- There are no docstrings anywhere; the code relies entirely on naming, and assumptions such as a non-empty list are never stated.
- `print_results` mixes presentation into an otherwise computational class, giving it two reasons to change.
- For a task this small, the class ceremony (`class, __init__, self`) delays the reader's arrival at actual logic.

Part Three: Code Modification

The Variation

Each sample was modified to implement the same two-part variation of the original task: (1) the report now includes a letter-grade distribution (A for 90 and above, B for 80 to 89, C for 70 to 79, D for 60 to 69, F below 60), and (2) the passing threshold is configurable instead of hardcoded at 70. Each modified program prints the report under the default threshold of 70 and again under a stricter threshold of 80 to demonstrate the new capability. The modifications are in `imperative_modified.py`, `functional_modified.py`, and `objectoriented_modified.py`, with every changed or added line tagged MODIFICATION. All three produce identical output.

How Each Structure Supported or Hindered the Change

Imperative: The single-pass loop made the distribution cheap to add in one sense: five new accumulators and one if/elif chain slotted straight into the existing loop, and everything still computes in one traversal. But every change landed inside the same function: the signature changed to carry the threshold, new mutable state joined the loop, and new lines joined the print block. Because every variable is in scope everywhere, I had to re-read the entire function to be confident the additions broke nothing unrelated. The fusion of computation and printing also means that demonstrating a second threshold requires re-running the entire report just to change one number.

Functional: Adding the distribution was the paradigm's best moment: `letter_for`, `grade_distribution`, and `format_distribution` are brand-new pure functions written without touching a single existing line, and the report grew by appending one lambda to the list in `create_analysis_functions`. The threshold, however, exposed the cost of purity. Because pure functions share no state, the parameter had to be threaded through the entire call chain (`is_passing`, `count_passing`, `calculate_pass_rate`, `create_analysis_functions`, and `generate_report` all changed signature) for what is conceptually a single change. One satisfying detail: the previously unused partial import finally earned its place, binding the threshold onto the predicate for `filter`. A genuine hindrance appeared in the distribution logic itself, where building a dictionary immutably with `reduce` is markedly harder to read than the equivalent loop.

Object-oriented: This was the smoothest modification. The threshold became constructor state: one new parameter with a default value, one attribute, and one changed comparison in

`get_pass_rate`. No other method signature moved, and existing callers kept working unchanged because of the default. The distribution arrived as a new public method plus a private helper without modifying any existing computation method, which is close to the open/closed principle in practice. The class also expresses something the other two versions cannot say as naturally: two analyzers with different thresholds coexisting as separate objects. The one friction point confirmed an earlier observation: `print_results` had to be edited again, because presentation is coupled to the class.

Part Four: Evaluation and Comparison

Maintainability of Each Sample (Based on the Modification Experience)

Imperative. Lowest. The change surface is always the whole function: every new requirement adds state to the same loop and lines to the same print block, so complexity accumulates in one place and the risk of touching unrelated logic grows with every edit. There is also no way to obtain one statistic without running the entire report.

Functional. Good for additive change, weaker for cross-cutting change. New computations are safe and isolated because pure functions cannot disturb existing behavior, but any parameter that multiple functions need ripples through every signature between the caller and the point of use.

Object-oriented. Best. Both halves of the variation were absorbed with the smallest, most localized edits: configuration went into the constructor once, the new feature arrived as a new

method, and default values kept every existing caller working. The object boundary contains change instead of spreading it.

Readability of Each Sample (Based on the Review and Modifications)

Imperative. The most immediately readable at its original size; any programmer can trace it without knowing Python idioms. But the modification visibly degraded it: the loop body now interleaves three unrelated concerns (sum/count, min/max, and letter binning), and this decline would continue with every added feature.

Functional. The highest cognitive load of the three. The reader must be comfortable with higher-order functions, lambdas, and the indirection of a list of functions before the simple task underneath becomes visible. Its genuine readability virtues, small named units and a clean computation/output split, survive the modification, but the threaded threshold parameter made every signature noisier.

Object-oriented. The most readable after change. The method names double as documentation, the new feature reads as one more entry in an existing interface, and the modification degraded its readability the least of the three.

Performance Expectations

All three versions are $O(n)$ and indistinguishable at ten grades; none has a meaningful advantage at this scale. The interesting differences are structural. The imperative version makes exactly one pass over the data, while the functional and object-oriented versions traverse the list once per statistic (six passes after the modification). Counterintuitively, fewer passes does not

mean faster in Python: the object-oriented version's per-statistic passes run inside C-implemented built-ins (sum, max, min), which typically outrun a single Python-level loop on large inputs. The functional version is the expected slowest, because it adds a Python-level lambda call per element plus intermediate list construction (filter into list into len), and the modified distribution rebuilds an immutable dictionary on every element. The practical conclusion: at realistic scales for this task, performance should not drive the team's choice; maintainability should.

Overall Maintainability Comparison

Ranked: object-oriented first, functional second, imperative third. The object-oriented version localizes change (constructor for configuration, new methods for new features) and preserves existing callers through defaults. The functional version is excellent when change means adding computations, but cross-cutting parameters force signature changes across the call chain. The imperative version concentrates every change in one growing function, which is manageable today and steadily worse with each requirement.

Overall Readability Comparison

The honest answer is that readability depends on the reader and the scale. For a thirty-line script read by a novice, the imperative version wins: it demands no idioms and reads top to bottom. As the codebase and team grow, the object-oriented version wins: its interface is navigable, self-documenting, and stable under change. The functional version demands the most background of the three and hides a simple task behind indirection, though its discipline of separating pure computation from output is the single best readability idea in any of the samples.

Recommendation

Based on this analysis, the object-oriented version is the strongest foundation for future development: it absorbed the requirements change with the smallest localized edits, supports per-instance configuration, and remains readable as it grows. The team should borrow one idea from the functional version as it evolves the code: keep pure computation separate from presentation, which in practice means eventually moving `print_results` out of the `GradeAnalyzer` class into a separate reporting function.

Program Output

```
~/Desktop/COMP-SCI/MASTERS/CS 660/MOD 1 05:17:08
> python3 imperative_modified.py && echo && python3 functional_modified.py && python3 objectoriented_modified.py
Grade Analysis Results (passing threshold: 70):
Total students: 10
Average grade: 86.6
Highest grade: 96
Lowest grade: 73
Pass rate: 100.0%
Grade distribution: A=4 B=4 C=2 D=0 F=0

Grade Analysis Results (passing threshold: 80):
Total students: 10
Average grade: 86.6
Highest grade: 96
Lowest grade: 73
Pass rate: 80.0%
Grade distribution: A=4 B=4 C=2 D=0 F=0

Grade Analysis Results (passing threshold: 70):
Total students: 10
Average grade: 86.6
Highest grade: 96
Lowest grade: 73
Pass rate: 100.0%
Grade distribution: A=4 B=4 C=2 D=0 F=0

Grade Analysis Results (passing threshold: 80):
Total students: 10
Average grade: 86.6
Highest grade: 96
Lowest grade: 73
Pass rate: 80.0%
Grade distribution: A=4 B=4 C=2 D=0 F=0

Grade Analysis Results (passing threshold: 70):
Total students: 10
Average grade: 86.6
Highest grade: 96
Lowest grade: 73
Pass rate: 100.0%
Grade distribution: A=4 B=4 C=2 D=0 F=0

Grade Analysis Results (passing threshold: 80):
Total students: 10
Average grade: 86.6
Highest grade: 96
Lowest grade: 73
Pass rate: 80.0%
Grade distribution: A=4 B=4 C=2 D=0 F=0
```

AI Usage Acknowledgment

Portions of this assignment were completed with assistance from Anthropic's Claude (via Claude Code). The tool was used to help analyze the provided code samples, draft the readability annotations, and implement the task variation across the three paradigms. I reviewed, ran, and verified all the code.

Anthropic. (2026). Claude [Large language model]. <https://claude.ai>